

A Certified Admission Gate for TSN: From Network-Calculus Equations to Code

Huyen-Trang Le^{*a}, Aakash Soni^b and Samia Bouzefrane^c

^aIBISC Laboratory, Université d'Évry Paris-Saclay, Évry-Courcouronnes, France

^bLyRIDS Laboratory, ECE Paris, Paris, France

^cCEDRIC Laboratory, Conservatoire National des Arts et Métiers (Cnam), Paris, France

This report was compiled on October 3, 2026

Abstract—This technical report documents how the problem of online admission control in Time-Sensitive Networking (TSN) is formulated and how its solution is structured in code. The problem is the following: in a running TSN with a set of admitted flows, decide for each new flow request whether it can be added without any admitted flow missing its worst-case deadline, and prove the decision. We formulate it on a model of 1 Gbit/s switches with fixed shortest-path (ECMP) routing, token-bucket flows and non-preemptive static-priority ports, and solve it with deterministic network calculus: a per-hop delay bound derived from the residual service of each priority class, burst propagation along each path, and Total Flow Analysis for the end-to-end bound. For each equation, the report shows the function and lines of the Python package `tsn_gate` that compute it. It then explains how the implementation is organised: the full and incremental admission gates, the certificate and its seven checking conditions, the packet-level simulator and the cross-check against the reference tool `panco`, with a two-flow example worked by hand. Every number produced by the code can thus be re-derived on paper.

Keywords—Time-Sensitive Networking, deterministic network calculus, admission control, static priority, certificates

CONTENTS

1 Purpose and Structure of the Code	1
1.1 Units and Notation	1
2 System Model	1
2.1 Network and Ports	1
2.2 Routing	2
2.3 Feed-Forward Check	2
2.4 Reference Topologies	2
2.5 Flows	2
2.6 Scheduling at a Port	2
3 Per-Port Delay Bound	2
3.1 Derivation	2
3.2 Implementation	3
4 Burst Propagation and Total Flow Analysis	3
4.1 Output Burst	3
4.2 End-to-End Bound	3
4.3 The TFA Pass	3
5 The Admission Gate	3
5.1 Interface and Invariant	3
5.2 Full Gate	4
5.3 Incremental Gate	4
6 Certificate and Independent Checker	4
6.1 Content of a Certificate	4
6.2 Checking Conditions	4

^{*}huyen.le@etud.univ-evry.fr (H.-T. Le), aakash.soni@ece.fr (A. Soni), samia.bouzefrane@lecnam.net (S. Bouzefrane)

Companion to the four-page note *A Certified Admission Gate for Online Reconfiguration of Time-Sensitive Networks*. Source code: <https://github.com/hazel260802/net-work-calculus>.

7 Worked Example	5
8 Simulator	5
9 Validation Against <code>panco</code>	5
10 Experiments and Results	5
10.1 Setup	5
10.2 Results	5
10.3 Tests	6
11 Scope and Limits of the Implementation	6
Reproduction	6

1. PURPOSE AND STRUCTURE OF THE CODE

TSN provides bounded latency on switched Ethernet [3, 6], but its guarantees rest on an analysis of the whole flow set. When flows are added at run time, each addition can increase the delay of flows already admitted, including flows on other paths that share a port. The gate implemented here answers one question per request: *after adding this flow, does every admitted flow still meet its deadline?* It answers with a verdict and with evidence, a certificate, that can be checked without trusting the analysis code.

The implementation is the Python package `tsn_gate` in the public repository [7]. Table 1 maps each module to the part of the mathematical model it implements; the rest of this report follows the same order.

Table 1. Modules of `tsn_gate` and the model they implement

Module	Role	Equations
<code>model.py</code>	Flows, network, routing, feed-forward check	(1)–(4)
<code>topologies.py</code>	Reference two-tier topologies	(5)
<code>workload.py</code>	Seeded flow requests	(6)
<code>analysis.py</code>	Per-port bound, burst propagation, TFA	(10)–(13)
<code>gate.py</code>	Full and incremental gates	(14)
<code>certificate.py</code>	Witnesses, digest, checker Co-C6	(15)–(16)
<code>simulator.py</code>	Packet-level sanity check	(19)

Validation code (`validation/panco_check.py`) and experiments (`experiments/`) sit next to the package.

1.1. Units and Notation

All quantities are in SI base units: bits, seconds and bit/s. A byte is the constant `BYTE = 8`, and the link rate is `GBPS = 1e9`. Table 2 lists the symbols used throughout and their names in the code.

2. SYSTEM MODEL

2.1. Network and Ports

The network is an undirected graph whose vertices are switches and end stations and whose edges are full-duplex links of rate C . Each direction of a link is an output *port*, served by the node that transmits on it. For a link set $\mathcal{E}$, the port set is

$$\mathcal{P} = \{(u, v) : \{u, v\} \in \mathcal{E}\} \cup \{(v, u) : \{u, v\} \in \mathcal{E}\}, \quad (1)$$

Table 2. Notation and corresponding code identifiers

Symbol	Meaning	Code
C	Link rate of every port	<code>net.C</code>
t_{proc}	Constant processing delay per hop	<code>net.t_proc</code>
b_f, r_f	Burst and rate of flow f	<code>f.b, f.r</code>
L_f	Maximum frame size of f	<code>f.L</code>
k_f	Priority class of f (0 highest)	<code>f.prio</code>
$D_f^{\max}$	End-to-end deadline of f	<code>f.deadline</code>
$\mathcal{P}_f$	Ordered ports on the route of f	<code>net.route()</code>
$b_f^{(i)}$	Input burst of f at its i -th hop	<code>bursts[f][i]</code>
B_H, R_H	Burst and rate sums of higher classes	<code>B_H, R_H</code>
B_k, R_k	Burst and rate sums of class k	<code>B_k, R_k</code>
L_{lo}	Largest frame of a lower class	<code>L_lo</code>
$d_k(p)$	Delay bound of class k at port p	<code>ClassBound.delay</code>
D_f	End-to-end bound of f	<code>e2e[f]</code>
$\tau(p)$	Topological index of port p	<code>net.topo_index[p]</code>

which `Network.ports()` returns sorted. Every port, including the egress port of a source end station, adds the constant processing delay t_{proc} to the bound of every frame it serves; the simulator applies the same convention (Section 8).

2.2. Routing

Routing is fixed: each source–destination pair (s, t) always uses the same path, and end stations never forward. `Network.route()` first computes, by breadth-first search from t , the hop distance $\delta(v)$ of every node v that can reach t through switches only. The path is then built from s by always moving to a neighbour one hop closer to t :

$$v_{j+1} = N_j[h(s, t) \bmod |N_j|], \quad N_j = \text{sort}\{v \sim v_j : \delta(v) = \delta(v_j) - 1\}, \quad (2)$$

where $h(s, t)$ is the CRC32 checksum of the string " $s \rightarrow t$ ". When several next hops are equally short, as between the core switches of a two-tier network, the hash spreads the pairs over them like Equal-Cost Multi-Path (ECMP) routing, while keeping the route of a pair fixed and reproducible across runs. Routes are cached in `_routes`.

```

1 def route(self, src: str, dst: str) -> tuple:
2     """
3     Path as a tuple of ports [(src, s1), (s1, s2), ..., (sk, dst)].
4
5     Shortest path; ties between equal-cost next hops (e.g. the spines of a
6     leaf-spine) are broken by a CRC32 hash of (src, dst), as in ECMP. The
7     route of a pair is therefore fixed and reproducible across runs.
8     """
9     key = (src, dst)
10    if key not in self._routes:
11        # hop distance to dst; end stations do not forward traffic
12        dist = {dst: 0}
13        q = deque([dst])
14        while q:
15            u = q.popleft()
16            for v in self.adj[u]:
17                if v not in dist and (v == src or v not in self.end_stations):
18                    dist[v] = dist[u] + 1
19                    if v != src:
20                        q.append(v)
21            if src not in dist:
22                raise ValueError(f"no route {src} -> {dst}")
23            h = zlib.crc32(f"{src}->{dst}".encode())
24            nodes = [src]
25            while nodes[-1] != dst:
26                u = nodes[-1]
27                nxt = sorted(v for v in self.adj[u] if dist.get(v) == dist[u] -
28                    ↳ 1)
29                nodes.append(nxt[h % len(nxt)])
30            self._routes[key] = tuple(zip(nodes[:-1], nodes[1:]))
31    return self._routes[key]
```

Code 1. Shortest-path routing with ECMP tie-breaking (`model.py`).

2.3. Feed-Forward Check

Total Flow Analysis (TFA) as implemented needs the ports to be visited in an order where every port comes after all ports that feed it. Because

routing is fixed, every flow that will ever be admitted follows one of the $|\mathcal{S}|(|\mathcal{S}| - 1)$ routes between end stations $\mathcal{S}$. The constructor therefore builds, once, the *port dependency graph*

$$G = (\mathcal{P}, E), \quad (p, q) \in E \iff p, q \text{ are consecutive on some route}, \quad (3)$$

and computes a topological order with Kahn's algorithm, breaking ties by port name so the order is deterministic. The result is the index

$$\tau : \mathcal{P} \rightarrow \{0, \dots, |\mathcal{P}| - 1\}, \quad (p, q) \in E \Rightarrow \tau(p) < \tau(q). \quad (4)$$

If G has a cycle, no such order exists and the constructor raises an error: the network is not feed-forward. A ring of six switches, for example, is rejected, whereas the two-tier topologies below pass because every route crosses at most one core switch.

2.4. Reference Topologies

`topologies.py` builds two-tier topologies: n_c core switches, n_e edge switches each linked to every core switch, and e end stations per edge switch. (In the code the function is `leaf_spine`, the data-centre name of this structure.) The number of ports is twice the number of links:

$$|\mathcal{P}| = 2(n_c n_e + n_e e). \quad (5)$$

The experiments use $(n_c, n_e, e) = (2, 4, 5)$, giving $2(8 + 20) = 56$ ports, and $(4, 8, 5)$, giving $2(32 + 40) = 144$ ports. A one-switch network with three end stations, `single_switch(3)`, is used for the hand-computed example of Section 7.

2.5. Flows

A flow f is a frozen data class `Flow(id, src, dst, b, r, L, prio, deadline)`. At its source it is constrained by the token-bucket arrival curve

$$\alpha_f(t) = b_f + r_f t, \quad t > 0, \quad (6)$$

meaning that over any interval of length t it sends at most $\alpha_f(t)$ bits. `Flow.validate()` enforces $r_f \geq 0$, $0 < L_f \leq b_f$, $k_f \geq 0$ and $D_f^{\max} > 0$. The workload generator `random_flows()` draws seeded requests between random pairs of end stations from the two profiles of Table 3; a flow sends one frame of L_f bits per period, so $r_f = L_f/\text{period}$, and its burst is one or two frames, $b_f = L_f \times n_{\text{burst}}$ with n_{burst} the frames per burst.

Table 3. Traffic profiles of `workload.py`

Parameter	Class 0 (control)	Class 1 (stream)
Probability	0.3	0.7
Frame size	64–256 B	500–1500 B
Period	0.25, 0.5, 1 ms	0.125, 0.25, 0.5, 1 ms
Frames per burst	1	1 or 2
Deadline	$\mathcal{U}(0.1, 0.5)$ ms	$\mathcal{U}(0.5, 2)$ ms

2.6. Scheduling at a Port

Every port uses non-preemptive static priority (NP-SP): frames are queued per class, FIFO within a class; when the link becomes idle, the head frame of the highest non-empty class is sent; a frame in transmission is never interrupted. The link serves at rate C with no latency, so its service curve is $\beta(t) = Ct$.

3. PER-PORT DELAY BOUND

3.1. Derivation

Consider class k at port p . Let $\mathcal{F}_p$ be the flows that cross p , and b_g^{in} the burst of flow g when it reaches p (its initial burst, increased by the upstream hops as explained in Section 4). The code forms three groups and their aggregates:

$$B_H = \sum_{g \in \mathcal{F}_p, k_g < k} b_g^{\text{in}}, \quad R_H = \sum_{g \in \mathcal{F}_p, k_g < k} r_g, \quad (7)$$

$$B_k = \sum_{g \in \mathcal{F}_p, k_g=k} b_g^{\text{in}}, \quad R_k = \sum_{g \in \mathcal{F}_p, k_g=k} r_g, \quad (8)$$

$$L_{lo} = \max_{g \in \mathcal{F}_p, k_g > k} L_g, \quad (9)$$

with $L_{lo} = 0$ when no lower class is present. The higher classes together are constrained by $B_H + R_H t$. Because transmission is not preempted, a class- k frame can also find one lower-priority frame already on the link, which blocks it for at most L_{lo}/C . Subtracting both from the link service gives the classic residual rate-latency service curve of class k [1, 2]:

$$\beta_k(t) = (C - R_H) \left[t - \frac{B_H + L_{lo}}{C - R_H} \right]^+. \quad (10)$$

Class k is FIFO, so its aggregate $B_k + R_k t$ is served by β_k and every class- k frame is delayed by at most the horizontal deviation between the two curves. Adding the processing delay gives the bound used by the code:

$$d_k(p) = \frac{B_H + L_{lo} + B_k}{C - R_H} + t_{\text{proc}} \quad \text{if } R_H < C \text{ and } R_H + R_k \leq C, \quad (11)$$

and $d_k(p) = \infty$ otherwise. The rate condition says that the class does not, on average, receive more traffic than its residual rate; when it fails, the queue can grow without bound and the port is unstable.

3.2. Implementation

`analyze_port()` (Code 2) implements (7)–(11) literally. Its input is the list of `(prio, b_in, r, L)` tuples of the flows at the port, and it returns one `ClassBound` per class present, holding $B_H, R_H, L_{lo}, B_k, R_k$ and $d_k(p)$.

```

1 def analyze_port(C: float, t_proc: float, entries) -> PortResult:
2     """Pure function. entries: iterable of (prio, b_in, r, L) for flows crossing
3     ↳ the port."""
4     by_prio = {}
5     for prio, b, r, L in entries:
6         by_prio.setdefault(prio, []).append((b, r, L))
7     prios = sorted(by_prio)
8     classes = {}
9     for k in prios:
10        hi = [x for p in prios if p < k for x in by_prio[p]]
11        lo = [x for p in prios if p > k for x in by_prio[p]]
12        B_H = math.fsum(b for b, _, _ in hi)
13        R_H = math.fsum(r for _, r, _ in hi)
14        L_lo = max((L for _, _, L in lo), default=0.0)
15        B_k = math.fsum(b for b, _, _ in by_prio[k])
16        R_k = math.fsum(r for _, r, _ in by_prio[k])
17        if R_H < C and R_H + R_k <= C:
18            delay = (B_H + L_lo + B_k) / (C - R_H) + t_proc
19        else:
20            delay = INF
21        classes[k] = ClassBound(k, B_H, R_H, L_lo, B_k, R_k, delay)
22    return PortResult(classes)

```

Code 2. Per-port NP-SP bound (11) (`analysis.py`).

Two implementation choices matter for the rest of the design:

- **Pure function.** The result depends only on C , t_{proc} and the multiset of entries. Both gates call this same function, which is what makes their results comparable bit for bit (Section 5.3).
- **Exactly rounded sums.** Sums use `math.fsum`, which returns the correctly rounded value of the exact sum. The result therefore does not depend on the order in which flows are listed, unlike a plain floating-point loop.

4. BURST PROPAGATION AND TOTAL FLOW ANALYSIS

4.1. Output Burst

A server that delays every bit of a flow by at most d cannot make the flow burstier than its input shifted by d : if the input satisfies $b + rt$, the output satisfies $\alpha(t + d) = (b + rd) + rt$ [1]. The input burst at the next hop is therefore

$$b_f^{(i+1)} = b_f^{(i)} + r_f d_{k_f}(p_i), \quad b_f^{(0)} = b_f, \quad (12)$$

where p_i is the i -th port of $\mathcal{F}_f$. `output_burst()` returns $b + rd$, or ∞ if $d = \infty$. The rate is unchanged.

4.2. End-to-End Bound

The end-to-end delay of f is at most the sum of its per-hop bounds:

$$D_f = \sum_{i=0}^{|\mathcal{F}_f|-1} d_{k_f}(p_i), \quad (13)$$

computed by `eze_bound()` with `math.fsum`. A flow set is *schedulable* when every port is stable for every class present and every flow meets its deadline:

$$\forall f : d_{k_f}(p) < \infty \quad \forall p \in \mathcal{F}_f \quad \text{and} \quad D_f \leq D_f^{\text{max}}. \quad (14)$$

`_violations()` in `gate.py` tests this condition flow by flow and returns a readable message for each failure: the first unstable hop, or the bound and the deadline that it exceeds.

4.3. The TFA Pass

`full_analysis()` (Code 3) evaluates (11) and (12) in one sweep over the ports in increasing $\tau(p)$:

1. build `members[p]`, the list of (flow, hop index) pairs crossing each port;
2. initialise $b_f^{(0)} = b_f$ for every flow;
3. for each port in topological order, call `analyze_port()` with the current input bursts, then write $b_f^{(i+1)}$ for every flow that continues;
4. sum the hop bounds of every flow with (13).

By (4), when port p is processed, every port upstream of it on any route has already been processed, so all the input bursts it needs are final. The pass costs $O(\sum_p K_p |\mathcal{F}_p|)$ operations, where K_p is the number of classes at p .

```

1 def full_analysis(net, flows: dict, paths: dict) -> Analysis:
2     """flows: fid -> Flow, paths: fid -> tuple of ports. Recomputes everything."""
3     ↳ =
4     members = {}
5     for fid, path in paths.items():
6         for i, p in enumerate(path):
7             members.setdefault(p, []).append((fid, i))
8     bursts = {fid: [flows[fid].b + [None] * (len(paths[fid]) - 1)] for fid in
9     ↳ flows}
10    port_results = {}
11    for p in sorted(members, key=net.topo_index.__getitem__):
12        res = analyze_port(net.C, net.t_proc,
13        ↳ [(flows[f].prio, bursts[f][i], flows[f].r, flows[f].L)
14        ↳ for f, i in members[p]])
15        port_results[p] = res
16        for f, i in members[p]:
17            if i + 1 < len(paths[f]):
18                bursts[f][i + 1] = output_burst(bursts[f][i], flows[f].r,
19                ↳ res.classes[flows[f].prio].delay)
20    eze = {f: eze_bound(flows[f], paths[f], port_results) for f in flows}
21    return Analysis(port_results, bursts, eze)
22
23 def output_burst(b: float, r: float, d: float) -> float:
24     """Output arrival curve of a server with delay bound d: alpha(t + d) = (b + r
25     ↳ t) + d if d < INF else INF
26     return b + r * d if d < INF else INF
27
28 def eze_bound(flow, path, port_results) -> float:
29     return math.fsum(port_results[p].classes[flow.prio].delay for p in path)

```

Code 3. TFA pass, burst propagation (12) and end-to-end bound (13).

5. THE ADMISSION GATE

5.1. Interface and Invariant

The gate holds the admitted set S . On a request f it analyses $S \cup \{f\}$ and accepts if and only if that set satisfies (14). The invariant is that *every flow in S meets its deadline under the model*. A request is rejected not only when f itself would miss its deadline, but also when its interference would make an already admitted flow miss its own.

Co is necessary. Omitting a flow from a certificate only lowers the aggregates seen by the others, so C1–C6 would still pass. Only the comparison with the flow database detects the omission; a unit test confirms that the checker rejects such a certificate only when the database is supplied.

7. WORKED EXAMPLE

The test file `tests/hand_calculation.py` fixes one switch `sw0`, three end stations, $C = 1$ Gbit/s and $t_{proc} = 0$, and two flows that share port (`sw0, es1`):

- f_1 : $es0 \rightarrow es1$, class 1, $b = L = 12\,000$ bit (1500 B), $r = 12$ Mbit/s;
- f_2 : $es2 \rightarrow es1$, class 0, $b = L = 800$ bit (100 B), $r = 0.8$ Mbit/s.

Hop 1 (source ports). Each flow is alone on its source port, so (11) reduces to $d = b/C$: $d_1 = 12\,000/10^9 = 12\,\mu\text{s}$ and $d_2 = 0.8\,\mu\text{s}$. By (12), the bursts entering the switch port are $b'_1 = 12\,000 + 12 \cdot 10^6 \times 12 \cdot 10^{-6} = 12\,144$ bit and $b'_2 = 800 + 0.8 \cdot 10^6 \times 0.8 \cdot 10^{-6} = 800.64$ bit.

Hop 2 (shared port). For class 0 (f_2): $B_H = R_H = 0$, $L_{lo} = 12\,000$ (one class-1 frame may be on the link) and $B_k = 800.64$, so

$$d_0 = \frac{0 + 12\,000 + 800.64}{10^9} = 12.80064\,\mu\text{s}. \quad (17)$$

For class 1 (f_1): $B_H = 800.64$, $R_H = 0.8$ Mbit/s, $L_{lo} = 0$ and $B_k = 12\,144$, so

$$d_1 = \frac{800.64 + 0 + 12\,144}{10^9 - 0.8 \cdot 10^6} = 12.955004\,\mu\text{s}. \quad (18)$$

End to end. $D_1 = 12 + 12.955004 = 24.955004\,\mu\text{s}$ and $D_2 = 0.8 + 12.80064 = 13.60064\,\mu\text{s}$. The gate's witnesses match these values to a relative error of 10^{-12} , and `panco`'s end-to-end analysis returns $13.600600\,\mu\text{s}$ for f_2 , the same value printed to six significant digits.

The same file checks the three ways a request can be rejected: (i) a class-0 flow with a $10\,\mu\text{s}$ deadline misses its own deadline; (ii) admitting f_2 raises D_1 from $12 + 12.144 = 24.144\,\mu\text{s}$ (alone) to $24.955\,\mu\text{s}$, so if f_1 has a deadline of $24.5\,\mu\text{s}$, f_2 is rejected although it would meet its own; (iii) a class-1 flow added behind a class-0 flow of 0.9 Gbit/s violates $R_H + R_k \leq C$ and is reported unstable.

8. SIMULATOR

`simulate()` is a discrete-event, packet-level simulator used as a sanity check: it must never observe a delay larger than the bound. Each flow sends frames of exactly L_f bits. With $n_f = \lfloor b_f/L_f \rfloor$ frames in the initial burst and a random offset $t_0 \in [0, L_f/r_f]$, frame j is created at

$$t_j = \begin{cases} t_0, & j < n_f, \\ t_0 + (j - n_f + 1)L_f/r_f, & j \geq n_f. \end{cases} \quad (19)$$

Up to time $t_0 + mL_f/r_f$ the flow has sent $(n_f + m)L_f \leq b_f + r_f \cdot mL_f/r_f$ bits, so the emission conforms to (6). Ports keep one FIFO per class, start the highest non-empty class whenever the link is idle, transmit for L/C without preemption, and pass the frame to the next port after t_{proc} . The worst delay of each flow over the horizon (10 ms in the experiments) is compared with D_f . Because offsets are random rather than adversarial, the simulator does not force the worst case: the ratio observed/bound is a sanity check, not a measure of tightness.

9. VALIDATION AGAINST PANCO

`validation/panco_check.py` compares the gate with `panco` [4, 5], an independent network-calculus tool, on five scenarios: the example of Section 7 and two seeds of 60 requests on each topology, with $t_{proc} = 0$ and times converted to microseconds to keep the linear programs well scaled.

- **Per hop.** For every port and class, `panco`'s `SpServer.residual()` builds the residual service curve from (B_H, R_H) and the lower-class frame sizes; its delay $T + B_k/R$ is compared with $d_k(p)$, and `TokenBucket.delay(d)` with the propagated burst (12).

- **End to end.** `panco`'s `SpNetwork` builds one FIFO network per class from residual servers, and `TfaLP` computes the TFA bounds as a linear program solved by `lp_solve`, independently of our propagation code.

The 481 hop delays and 638 output bursts agree to a relative difference of at most 2.5×10^{-16} , the floating-point precision; the 242 end-to-end bounds agree to 2.9×10^{-6} , the precision at which `lp_solve` prints its results (Fig. 1).

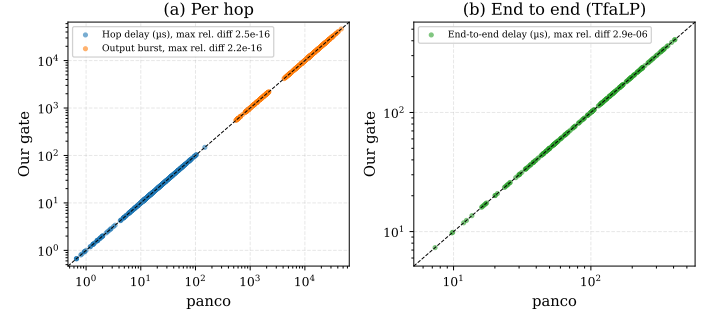


Figure 1. Bounds of the gate against `panco`, per hop and end to end.

10. EXPERIMENTS AND RESULTS

10.1. Setup

`experiments/benchmark_gate.py` runs, on each topology and for 10 seeds, a greedy proposer that submits 1,000 random requests in arrival order and keeps every accepted flow. Both gates process the same sequence with $C = 1$ Gbit/s and $t_{proc} = 1\,\mu\text{s}$; every admitted set of the incremental gate is then simulated for 10 ms. Results are written to `results/requests.csv` and `results/simulation.csv`, and `plot_figures.py` draws the figures. All workloads and simulations are seeded; only the timing columns depend on the machine (single-threaded CPython 3.14).

10.2. Results

Table 4 summarises the measurements.

Table 4. Results over 10 seeds $\times$ 1,000 requests

Quantity	56 ports	144 ports
Accepted requests (all)	first 50	first 200
Admitted after 1,000	150 ± 14	342 ± 22
Median time, full gate	3.10 ms	7.32 ms
Median time, incremental	1.62 ms	3.33 ms
Speed-up of incremental	1.97×	2.24×
Ports recomputed (median)	24 of 56	49 of 144

Over all runs both gates give identical verdicts and bit-identical bounds. No simulated delay exceeds its bound over 4,915 admitted flows.

The incremental gate's advantage grows with the network because the affected region becomes a smaller fraction of the ports (Fig. 2). In simulation (Fig. 3), the median ratio of observed delay to bound is 0.19 for class 0 and 0.10 for class 1, with maxima of 0.71 and 0.43. Admission saturates because TFA is pessimistic (Fig. 4); class 0 is accepted slightly more often, since its priority shields it from most interference.

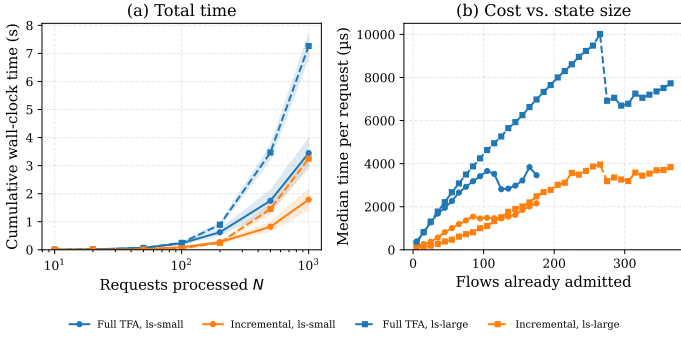


Figure 2. Run time of the full and incremental gates: (a) cumulative time; (b) median time per request against the number of admitted flows.

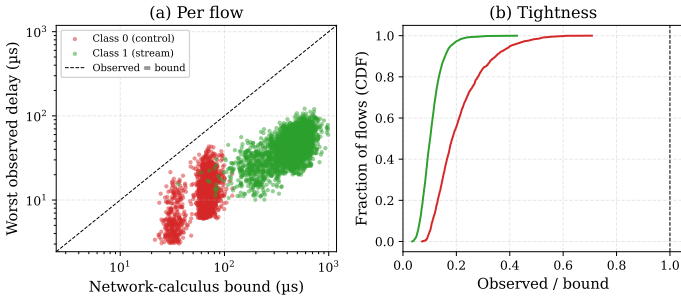


Figure 3. Worst simulated delay against the bound for 4,915 admitted flows.

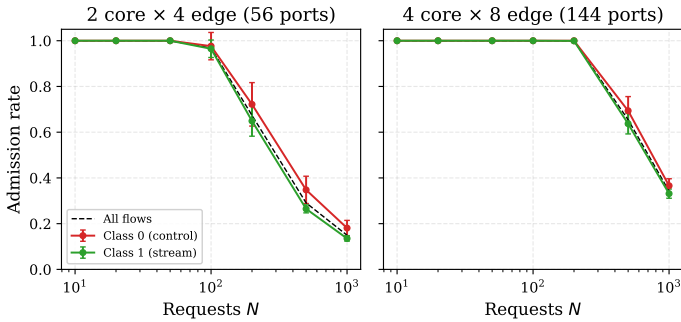


Figure 4. Fraction of the first N requests accepted, per class (mean $\pm$ std over 10 seeds).

10.3. Tests

The 18 unit tests (`python -m pytest -q` in `src/`) cover the hand-computed example to 10^{-12} , the three rejection cases, full-versus-incremental equality with exact float comparison on both topologies, the locality and validity of incremental certificates, the checker’s rejection of a tampered certificate (a halved hop delay, caught by C5; an omitted flow, caught by C0), and the simulator.

11. SCOPE AND LIMITS OF THE IMPLEMENTATION

- **Model.** One link rate, fixed shortest-path routing, feed-forward port dependencies and strict NP-SP. Credit-based and time-aware shapers, frame preemption and clock effects are not modelled; Ethernet overheads must be folded into L_f , b_f and r_f .
- **Analysis.** TFA bounds each port separately. It is sound but pessimistic compared with analyses that pay bursts only once along a path [2]; pessimism lowers the admission rate but never admits an unsafe flow.
- **Arithmetic.** The checker uses floating point with the tolerance (16), which the soundness argument does not account for; exact rational arithmetic would remove this gap.

- **Operations.** Only flow additions are implemented: no removal, no batched changes, and only a greedy proposer.

REPRODUCTION

All the source code is available at github.com/hazel260802/network-calculus. From the `src` directory, run:

- `pip install -r requirements.txt`
- `python -m pytest -q`
- `python experiments/benchmark_gate.py`
- `python experiments/plot_figures.py`

The panco check needs panco at commit `f035ccc5`, set as `PANCO_PATH`, and `lp_solve` for the end-to-end mode:

- `python validation/panco_check.py -mode hop`
- `python validation/panco_check.py -mode e2e`

REFERENCES

- [1] J.-Y. Le Boudec and P. Thiran, *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet* (Lecture Notes in Computer Science). Springer, 2001, vol. 2050. DOI: [10.1007/3-540-45318-0](https://doi.org/10.1007/3-540-45318-0)
- [2] A. Bouillard, M. Boyer, and E. Le Corronc, *Deterministic Network Calculus: From Theory to Practical Implementation*. Wiley-ISTE, 2018. DOI: [10.1002/9781119440284](https://doi.org/10.1002/9781119440284)
- [3] L. Lo Bello and W. Steiner, “A perspective on IEEE time-sensitive networking for industrial communication and automation systems”, *Proceedings of the IEEE*, vol. 107, no. 6, pp. 1094–1120, 2019. DOI: [10.1109/JPROC.2019.2905334](https://doi.org/10.1109/JPROC.2019.2905334)
- [4] A. Bouillard, *Panco: Performance analysis with network calculus and optimization*, Commit f035ccc5, 2022. [Online]. Available: <https://github.com/anne-bou/panco>
- [5] A. Bouillard, “Trade-off between accuracy and tractability of network calculus in FIFO networks”, *Performance Evaluation*, vol. 153, p. 102250, 2022. DOI: [10.1016/j.peva.2021.102250](https://doi.org/10.1016/j.peva.2021.102250)
- [6] IEEE, *IEEE standard for local and metropolitan area networks—bridges and bridged networks*, IEEE Std 802.1Q-2022, 2022. DOI: [10.1109/IEEESTD.2022.10004498](https://doi.org/10.1109/IEEESTD.2022.10004498)
- [7] H.-T. Le, *Network-calculus: Certified admission gate for TSN*, 2026. [Online]. Available: <https://github.com/hazel260802/network-calculus>